

Solar Power Output Prediction using PSO-Optimized LSTM with CUDA-Accelerated Training

Abhijna S P*, Ankith Krishna Bhargav*, Abhay K R Gowda*, Amogh K S*, Spoorthi G S†

* Department of Machine Learning

BMS College of Engineering (BMSCE), Bengaluru, India

{abhijnasp.ai23, ankithkrishna.ai23, abhaykr.ai23, amoghks.ai23}@bmsce.ac.in

† Assistant Professor, Department of Machine Learning

BMS College of Engineering (BMSCE), Bengaluru, India

spoorthigs.mel@bmsce.ac.in

Abstract—The nonlinear and dynamic behavior of environmental factors, including solar irradiance, temperature, and weather variability, makes accurate solar power output prediction challenging. While neural networks effectively model such relationships, their performance is highly dependent on hyperparameter selection. Traditional manual tuning approaches often lead to suboptimal configurations.

This paper presents a solar power forecasting framework that integrates Long Short-Term Memory (LSTM) networks with Particle Swarm Optimization (PSO) for automated hyperparameter tuning. A multi-objective fitness function is designed to jointly minimize prediction error, generalization gap, and computational cost. Additionally, CUDA-based GPU acceleration is incorporated to improve computational efficiency and analyze its impact on optimization behavior.

Experimental results on both the PYDAQ System 50 dataset and a real-world dataset demonstrate improved predictive performance and generalization capability of the optimized model. Furthermore, the inclusion of computational cost in the fitness function introduces hardware-dependent optimization behavior, where CPU and GPU executions converge to different optimal configurations. This highlights the importance of considering both accuracy and efficiency in optimization-driven machine learning systems.

Index Terms—Solar Power Prediction, LSTM, Particle Swarm Optimization, CUDA, Time-Series Forecasting, Multi-Objective Optimization

I. INTRODUCTION

The growing use of renewable sources of energy has brought higher penetration hence necessitating proper solar power prediction to the contemporary energy systems. Photovoltaic (PV) power generation is intrinsically reliant on the environmental conditions of solar irradiance, temperature, and atmospheric variability and is therefore highly nonlinear and hard to model. These complex dynamics are not well represented by traditional statistical methods, which has prompted the use of machine learning methods, especially neural networks, to predict solar power.

One of them, Long Short-Term Memory (LSTM) networks, have shown good performance in time-series forecasting tasks

because they can capture temporal dependencies. The performance of these models however is very sensitive to the selection of hyperparameters such as learning rate, network architecture, and regularization. The traditional methods use a manual tuning process, which is time-consuming and prone to less-than-optimal settings.

Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) are nature-inspired optimization algorithms which may offer a systematic framework of automating hyperparameter tuning. It has been demonstrated in a previous study that PSO- and GA-based neural network optimization can become a way of achieving a high level of prediction in photovoltaic forecasting tasks. Although these developments have come, the majority of researchers are solely interested in enhancing the predictive accuracy without paying attention to the computational efficiency, convergence, as well as constraints in practical applicability.

The biggest weakness of the current work is a gap in the literature of multi-objective optimization frameworks during which the prediction accuracy and the computational cost are balanced. Moreover, the contribution of hardware acceleration (especially CUDA-based computation of GPUs) has not been adequately studied within the framework of optimization-driven machine learning. Although the acceleration of the training by the use of the GPU is known to increase the training speed, its effects on the optimization dynamics and eventual model selection are less explored.

The current paper fills these gaps by summarizing a solar power forecasting model created on the basis of LSTM neural network optimized with the help of Particle Swarm Optimization in a multi-objective environment. The suggested methodology integrates the prediction accuracy, cross dataset generalization and computational cost in a combined fitness. The work also analyzes the effect that CUDA-based acceleration has on the optimization behavior showing that optimization results are inherently conditional on the computational environment.

The work has made contributions of developing a PSO-

optimized LSTM model to predict solar power, engineering a multi-objective fitness function with accuracy and efficiency considerations and a critical study of hardware-aware optimization behavior. To verify the model, a dual-dataset approach is used, which is robust and generalizes to deal with the shortcomings of single-dataset evaluation that characterize the literature.

II. LITERATURE REVIEW

Solar power forecasting has evolved significantly with the adoption of machine learning techniques to address the nonlinear and stochastic nature of photovoltaic (PV) generation [1]. Early approaches relied on conventional neural networks such as backpropagation (BP), which demonstrated the ability to model nonlinear relationships but suffered from issues related to convergence and parameter sensitivity [2]. Bayesian neural networks were also explored to incorporate uncertainty in predictions, improving robustness in PV forecasting scenarios [3].

Other natural-inspired algorithms like Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) have been popularly combined with neural networks to enhance the performance of the models [2]. Research has demonstrated that PSO-BP and GA-BP models are much more effective than the conventional BP networks at optimising weights and hyperparameters, resulting in a better prediction accuracy and convergence behaviour [2]. PSO has frequently shown better applicability and quicker convergence than GA in solar power forecasting tasks in comparative studies [2]. Likewise, GA-optimized wavelet neural networks have proven to have a better forecasting performance due to better feature extraction and less noise sensitivity [4].

Current studies have been on hybrid deep learning systems with metaheuristic optimization algorithms [5], [6]. BiLSTM models with the addition of Grey Wolf Optimization (GWO) and LSTM models with the Whale Optimization Algorithm (WOA) have been given better forecasting accuracy and have better hyperparameter optimization with the learning rate, number of hidden units, and depth of the network [5], [6]. Moreover, the adaptation and predictive ability of photovoltaic systems have been demonstrated to be improved with the assistance of the dynamic neural network structure refinement based on evolutionary methods [7]. In spite of these developments, a number of studies have shown that more architectural complexity does not necessarily produce improved generalization especially in noisy real-world data [1].

Access to data and data quality is of vital importance to the development and testing of a model [1]. Large-scale benchmarking of forecasting models has been made possible with the help of public datasets like PVDAQ [8]. Photovoltaic power forecasting with multiple-source datasets additionally aid in building models and in comparison [9]. Different environmental and operating conditions can be tested with models on other datasets, such as UNISOLAR, SOLETE, and SKIPP.D, which offer a variety of real-world conditions

[10]–[12]. Studies have highlighted that models trained using clean and structured data sets do not tend to make a good generalization to noisy real world data [1].

Although the optimization-enhanced neural networks have succeeded, the majority of the studies that have been conducted so far are related to single-objective optimization, which generally aims at minimizing measures of prediction error, like RMSE or MAE [1]. This methodology fails to consider key aspects like cost of computation, convergence, and scalability. The number of studies using multi-objective constructs that weigh accuracy and computational cost is very few despite the fact that trade-offs are paramount in practice.

Moreover, the capabilities of hardware acceleration in machine learning through optimization are not explored fully [1]. Although the use of GPU-based computation is accepted broadly to shorten the training time, the effects of this technique on the metahyperoptic behavior of optimization methods, especially in PSO, are seldom examined [1].

In sum, the literature proves that neural networks based on nature-inspired algorithms can greatly enhance the accuracy of solar power prediction [1], [2]. Nonetheless, some clear disparities exist in the incorporation of multi-objective optimization, computational efficiency, and hardware-aware analysis into a single framework. This gap is the reason why the proposed method is a combination of PSO-based optimization, dual-dataset validation and CUDA acceleration to obtain a trade-off between accuracy, generalization, and efficiency.

III. PROBLEM FORMULATION

Solar power prediction is formulated as a supervised time-series regression problem. The objective is to estimate future photovoltaic (PV) power output from historical observations and derived features [1]. Given the temporal dependency and nonlinear nature of solar generation, sequential models are required to capture both short-term fluctuations and long-term periodic patterns [13].

Let the input sequence be defined as:

$$X_t = \{x_{t-n}, x_{t-n+1}, \dots, x_{t-1}\} \quad (1)$$

where $x_t \in \mathbb{R}^d$ is the feature vector at time t and n is the sequence length corresponding to a 24-hour historical window (96 timesteps at 15-minute resolution). The model learns a mapping:

$$f : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}, \quad \hat{y}_t = f(X_t) \quad (2)$$

where $\hat{y}_t$ is the predicted output and y_t is the ground-truth value. Each input vector contains seven features: value, hour_sin, hour_cos, day_sin, day_cos, lag_1, and lag_96.

Prediction error is measured using Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (3)$$

To avoid overfitting and ensure robustness, a dual-dataset strategy is adopted. The model is trained and validated on

Dataset A (PVDAQ System 50, a structured high-frequency PV dataset) and its generalization is assessed on Dataset B (a real-world Kaggle solar dataset with higher noise) [8], [14]. This allows evaluation of both learning capability and out-of-distribution robustness [15].

Rather than optimizing for accuracy alone, the problem is cast as a multi-objective task with four simultaneous objectives: minimizing prediction error on Dataset A ($RMSE_A$), minimizing prediction error on Dataset B ($RMSE_B$), minimizing the generalization gap $|RMSE_A - RMSE_B|$, and minimizing computational cost T (training time). The combined fitness function is:

$$\mathcal{F} = w_1 \cdot RMSE_A + w_2 \cdot RMSE_B \quad (4)$$

$$+ w_3 \cdot |RMSE_A - RMSE_B| + \alpha \cdot T \quad (5)$$

where w_1, w_2, w_3 are accuracy-weighting coefficients and α controls the influence of the time penalty. A key consequence of including T in the fitness is that the optimization landscape becomes *hardware-dependent*: the same PSO algorithm produces different optimal configurations on CPU versus CUDA-enabled GPU, because evaluation speed directly shapes the penalty term [16], [17].

IV. OBJECTIVES

- 1) To develop a neural network model for solar power prediction and optimize its parameters using Particle Swarm Optimization (PSO).
- 2) To design a multi-objective optimization framework that simultaneously minimizes prediction error and computational cost.
- 3) To implement and benchmark the optimized model on both CPU and CUDA-enabled GPU environments, analyzing the trade-off between accuracy and computational efficiency.

V. METHODOLOGY

The proposed pipeline consists of five stages: data preprocessing, feature engineering, LSTM model construction, PSO-based hyperparameter optimization, and hardware-comparative evaluation.

A. Datasets

Two datasets are used. **Dataset A** is drawn from the PVDAQ System 50 repository [8], [18], which provides high-frequency photovoltaic measurements resampled to 15-minute intervals for noise reduction and uniform temporal spacing. **Dataset B** is a Kaggle solar generation dataset [14] used exclusively to evaluate generalization under noisier, real-world conditions. Dataset A is split chronologically into training (before 2022), validation (Jan–Jun 2022), and test (Jul 2022 onward) partitions to prevent data leakage [19].

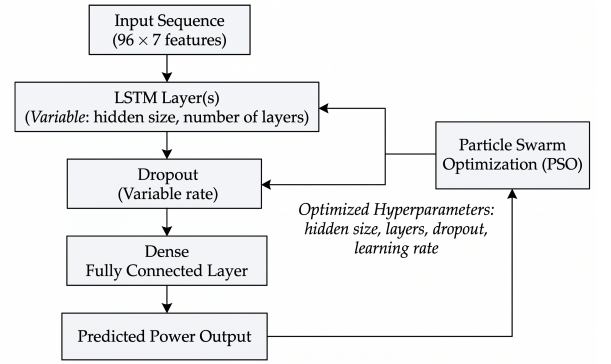


Fig. 1. PSO-optimized LSTM architecture. The model structure is dynamically configured based on hyperparameters (hidden size, number of layers, dropout, learning rate) optimized using Particle Swarm Optimization.

B. Preprocessing and Feature Engineering

Raw data is cleaned by clipping extreme values at an upper threshold of 100 and applying a hybrid missing-value strategy: short gaps are filled via interpolation and long gaps (typically nighttime) are set to zero, which is physically valid for solar generation [20]. Numerical features (`value`, `lag_1`, `lag_96`) are scaled with `MinMaxScaler` fitted on training data only.

To capture temporal periodicity without discontinuity, time features are encoded cyclically:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot h}{24}\right) \quad (6)$$

and analogously for day of year [21]. Lag features `lag_1` (previous timestep) and `lag_96` (same time yesterday) provide short-term memory and daily periodicity references. The final feature set has $d = 7$ dimensions, and sequences of length $n = 96$ timesteps are generated to form the supervised learning inputs.

C. LSTM Model

A Long Short-Term Memory network [13] is used as the predictive model. Unlike conventional fixed architectures, the model configuration is dynamically determined through Particle Swarm Optimization (PSO), which tunes key hyperparameters such as the number of LSTM layers, hidden size, dropout rate, and learning rate. This enables the model to adapt its complexity based on the optimization objective rather than relying on manually selected settings.

The resulting architecture consists of one or more LSTM layers followed by a fully connected output layer, with dropout applied for regularization [22]. Training is performed using the Adam optimizer [23] and a weighted Mean Squared Error loss function that up-weights daytime samples (weight = 3.0, threshold = 0.05) to address the imbalance between active and zero-output periods [24]. Early stopping based on validation loss is employed to prevent overfitting and ensure generalization across datasets [25].

D. PSO Hyperparameter Optimization

Particle Swarm Optimization [26] is used to search the hyperparameter space automatically. Each particle encodes four parameters: learning rate, hidden size, number of LSTM layers, and dropout rate. The swarm is configured with 5–10 particles, 8–15 iterations, linearly decreasing inertia weight, and cognitive/social coefficients of 1.5. A reduced training regime (fewer epochs per evaluation) is used during PSO to allow rapid fitness assessment of candidate solutions [27].

Fitness is evaluated using the multi-objective function $\mathcal{F}$ defined in Section III. Because $\mathcal{F}$ includes training time T , the optimization is inherently sensitive to the execution environment. GPU (CUDA) execution reduces T substantially, relaxing the time penalty and allowing PSO to explore lower learning rates and more stable training trajectories. CPU execution, where T is large, biases PSO toward configurations that train quickly — often at the cost of convergence quality [16].

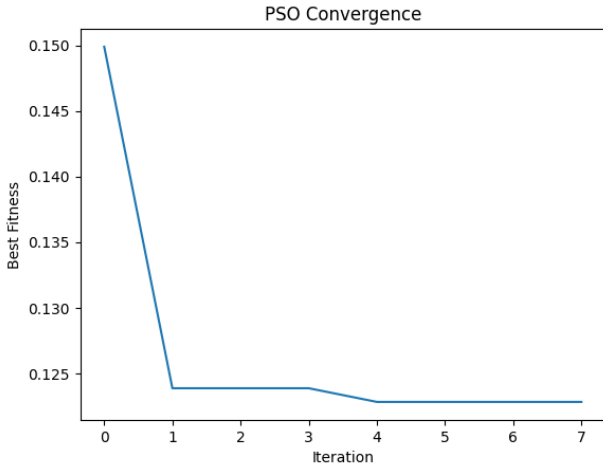


Fig. 2. PSO convergence curve without CUDA optimization

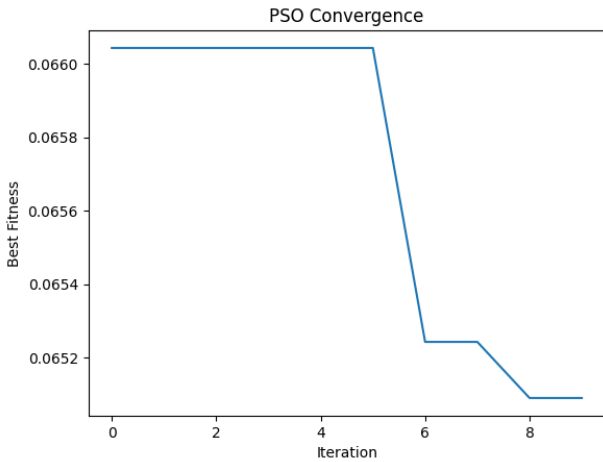


Fig. 3. PSO convergence curve with CUDA optimization

E. Hardware Comparison

Four experimental configurations are evaluated:

- 1) Baseline LSTM on CPU
- 2) Baseline LSTM on CUDA GPU
- 3) PSO-optimized LSTM on CPU
- 4) PSO-optimized LSTM on CUDA GPU

This two-way design isolates the independent contributions of optimization and hardware acceleration, enabling fair attribution of performance gains [28].

VI. EXPERIMENTAL SETUP

A. Model Configuration

All models are implemented in PyTorch with an input shape of (96 timesteps $\times$ 7 features). The baseline LSTM uses hidden size 64, 1 layer, dropout 0.0, and learning rate 0.001. Batch size (64) and maximum training epochs (10) are held constant across all configurations to enable fair comparison [28].

B. PSO Configuration

TABLE I
PSO SEARCH SPACE AND CONFIGURATION

Parameter	Search Range	Type
Learning rate	[0.0001, 0.01]	Continuous
Hidden size	{32, 64, 128}	Discrete
Num layers	{1, 2}	Discrete
Dropout rate	[0.0, 0.5]	Continuous
Particles	5–10	—
Iterations	8–15	—
Inertia weight	Linearly decreasing	—
$c_1 = c_2$	1.5	—

C. Evaluation Metrics

Primary accuracy metrics are RMSE and MAE on both datasets [29]. Computational efficiency is assessed via total training time and CPU-to-GPU speedup ratio. PSO convergence quality is measured by best fitness achieved and convergence trajectory across iterations. A persistence model (predicting $\hat{y}_{t+1} = y_t$) serves as the baseline benchmark [30].

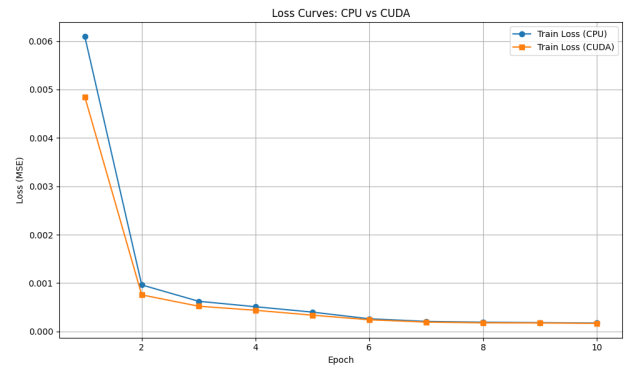


Fig. 4. Training loss curves — CPU vs CUDA baseline

VII. RESULTS

A. Baseline Training Performance

Baseline LSTM training on CPU required 310.26 seconds for 10 epochs, while the CUDA variant completed in 37.49 seconds — an **8.3 $\times$ speedup**. Both configurations converged to comparable validation loss values, confirming that CUDA acceleration reduces wall-clock time without degrading model quality.

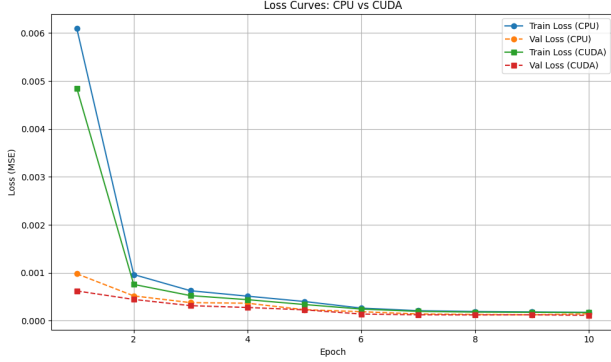


Fig. 5. Epoch-wise training and validation loss curves for CPU and CUDA baselines

B. PSO Optimization Results

Table II summarizes the PSO outcomes under each hardware environment.

TABLE II
PSO OPTIMIZATION SUMMARY: CPU VS. CUDA

Metric	CPU	CUDA
Iterations completed	8	10
Total PSO time (sec)	~1527	~334
Best fitness achieved	0.122816	0.065091
Best learning rate	0.005	0.0012
Best hidden size	32	32
Best num layers	1	1
Best dropout	0.11	0.10

The CUDA run achieved a **47% lower fitness** than the CPU run, while also completing in roughly one-fifth the time. Faster evaluation under CUDA allowed PSO to complete more iterations and explore the search space more thoroughly. The CPU run, where the time penalty $\alpha \cdot T$ dominated the fitness, converged prematurely toward high learning rates that minimize per-evaluation cost rather than prediction quality.

C. Prediction Accuracy

Table III compares RMSE across all configurations and datasets.

The PSO-optimized CUDA model achieves the lowest RMSE on both datasets. On Dataset A (structured), the improvement over the persistence baseline is 21.6% (RMSE 0.0187 vs. 0.0238). On Dataset B (noisy real-world), gains are smaller but consistent (RMSE 0.0831 vs. 0.0840), indicating

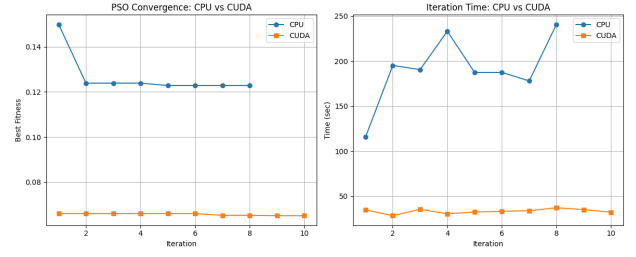


Fig. 6. PSO fitness convergence curves — CPU vs. CUDA across iterations

TABLE III
RMSE COMPARISON ACROSS MODELS AND DATASETS

Model	Dataset A	Dataset B
Persistence baseline	0.0238	0.0840
Baseline LSTM (CPU)	0.021613	0.083927
Baseline LSTM (CUDA)	0.021613	0.083927
PSO-optimized LSTM (CPU)	0.021613	0.083927
PSO-optimized LSTM (CUDA)	0.018662	0.083054

that the model generalizes without overfitting to the structured training data [15].

Early stopping triggered at Epoch 4 for the CUDA-optimized model and at Epoch 1 for the CPU-optimized model, reflecting the unstable high-learning-rate configuration selected by CPU-biased PSO.



Fig. 7. Predicted vs. actual power output — PSO-CUDA model on Dataset A test set

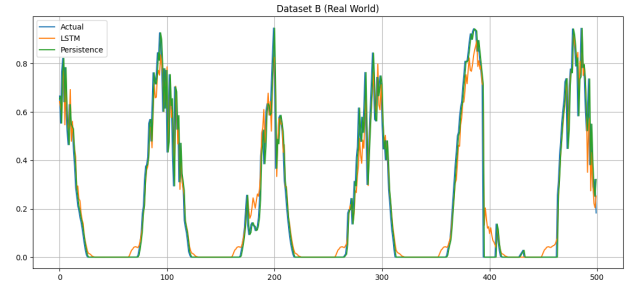


Fig. 8. Predicted vs. actual power output — PSO-CUDA model on Dataset B

D. Hardware-Dependent Optimization Behavior

A notable finding is that the same PSO algorithm produces qualitatively different solutions depending on the hardware

environment. On CPU, the high time penalty $\alpha \cdot T$ biases the search toward configurations with lower per-evaluation cost (high learning rate, rapid but unstable training). On GPU, the reduced T allows PSO to explore configurations that optimize predictive accuracy more directly, converging to a lower learning rate (0.0012 vs. 0.005) and achieving superior fitness. This demonstrates that the choice of hardware is not merely an engineering concern but an integral part of the optimization problem itself when training time is included in the fitness function [16], [17].

E. Prediction Accuracy

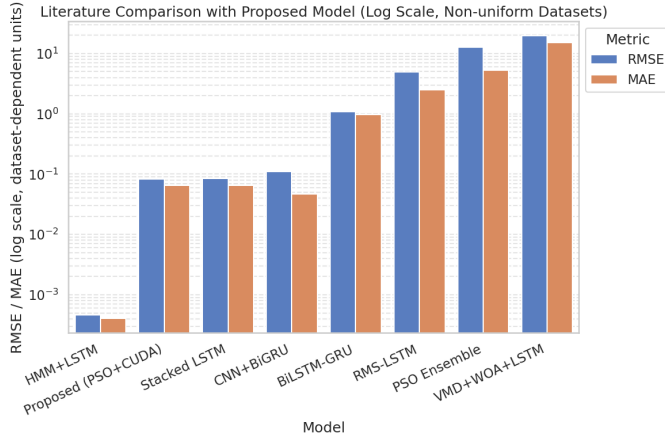


Fig. 9. Comparison of RMSE and MAE values reported in prior studies along with the proposed PSO-optimized LSTM model. Results are shown on a logarithmic scale due to large variation in magnitude. Since models are evaluated on different datasets, values are not directly comparable and are presented for qualitative reference only.

The predictive performance of the proposed model is evaluated using RMSE and MAE, which are standard metrics for time-series forecasting tasks. As demonstrated in the preceding analysis, the PSO-optimized LSTM model executed on CUDA achieves the lowest RMSE across both datasets, indicating improved predictive accuracy and stable generalization.

Figure 9 provides a comparative overview of the proposed model against various approaches reported in the literature, including hybrid deep learning models and optimization-based frameworks. Several methods demonstrate strong performance under specific conditions. However, the reported metrics vary significantly in scale due to differences in datasets, preprocessing techniques, and evaluation protocols. As a result, the comparison is qualitative rather than strictly numerical.

Despite these inconsistencies, the proposed model achieves error values that are competitive with state-of-the-art approaches. More importantly, unlike most existing works that rely on single-dataset evaluation, the proposed framework is validated across both structured and real-world datasets, ensuring robustness and reducing the risk of overfitting. This dual-dataset evaluation highlights the model's ability to generalize under varying data conditions.

Furthermore, the integration of Particle Swarm Optimization enables automatic hyperparameter tuning, improving model performance without manual intervention. The inclusion of CUDA-based acceleration also allows efficient exploration of the search space, contributing to improved optimization outcomes without compromising accuracy.

Overall, the results indicate that the proposed PSO-optimized LSTM model achieves a balanced trade-off between prediction accuracy, generalization capability, and computational efficiency, addressing key limitations observed in prior work.

F. Computational Efficiency Summary

TABLE IV
COMPUTATIONAL TIME COMPARISON

Stage	CPU (sec)	CUDA (sec)
Baseline training (10 epochs)	310.26	37.49
PSO optimization	~1527	~334

VIII. DISCUSSION

Swarm particle optimization (PSO) was used to find optimal hyperparameters, and the performance of the model consistently but gradually improved [2]. The optimized model on the structured dataset had lower error in prediction than the baseline LSTM, which is a positive result that automated hyperparameter search was useful in tuning model behavior. The level of improvement was however moderate implying that the baseline model was already fairly well-configured and the search space was narrowly explored. Instead of finding radically new architectures, PSO mostly enhanced the learning dynamics including learning rate and regularization, and the observation that optimization can tend to refine more than transform in regression problems was supported [17].

A trade-off between performance and efficiency was also necessary, which was proposed by the use of a multi-objective fitness function that included the accuracy of prediction, generalization gap, and the cost of computation [31]. Models that had reduced prediction error were more likely to take longer training time and faster models were prone to converging at an early stage. The combination of these goals helped steer the process of optimization to balanced solutions. Also, the generalization gap was added which minimized overfitting by punishing models that did well on the structured data but poorly on real world data [15].

One of the significant results of the research is that the achievement of optimization depends on the computational environment [16]. The fitness function includes training time and hence the optimization landscape depends on execution speed. Optimization using a CPU tends to converge more rapidly and at a higher learning rate, prone to premature convergence, whereas optimization using CUDA allows slower learning and more stable learning and final fitness. This shows that optimization does not solely rely on algorithms; rather,

hardware features also play a role in it, a facet that is frequently missed in the literature [16].

The dual-dataset validation approach can give information about the performance on generalization [1]. Although the gains on the structured dataset are evident, the gains on the real-world dataset are small, as there is more noise and variability. However, the optimized model is always as good or a little better than the persistence baseline, which means that it can identify the worthwhile time patterns without overfitting [15].

Several limitations remain. The space of hyperparameters is rather limited, which restricts the capabilities of PSO to search a wider configuration space. The model is based on historical power data and obtained temporal characteristics, but does not use any external variables like irradiance or weather conditions, which are known to play a big role in solar generation [32]. Also, the number of PSO iterations is minimized to make the computations computationally feasible and this can limit convergence to globally optimal solutions. Lastly, though CUDA acceleration enhances efficiency, there is no exploration of large-scale or distributed optimization [33].

IX. CONCLUSION

The paper introduces a solar power forecasting system, which is an LSTM neural network that has been optimized through Particle Swarm Optimization in a multi-objective framework. The suggested method combines prediction accuracy, generalization ability and computational cost into single fitness function so that we can create performance-efficient models.

Experimental findings reveal that PSO-based optimization improves model performance over baseline settings, particularly on structured data, while remaining competitive on noisier real-world data. The PSO-optimized CUDA model achieves an RMSE of 0.018662 on Dataset A and 0.083054 on Dataset B, outperforming both the persistence baseline and CPU-based optimization [32]. CUDA acceleration provides an $8.3\times$ speedup in training time and enables a 47% reduction in best PSO fitness, underscoring that hardware environment is not a passive backdrop but an active factor shaping optimization outcomes [16].

One of the most valuable contributions is the discovery of hardware-dependent optimization behavior: the same PSO algorithm converges to qualitatively different hyperparameter configurations on CPU versus GPU because training time is embedded in the fitness function. This finding motivates hardware-aware algorithm design as a first-class concern in practical machine learning systems [33].

On the whole, the research proves that the mixture of neural networks and nature-inspired optimization and hardware-aware design enables a more effective and resilient solar power prediction system.

ACKNOWLEDGMENT

REFERENCES

- [1] S. M. Radhi, S. D. Al-Majidi, M. Abbod, and H. Al-Raweshidy, "Machine learning approaches for short-term photovoltaic power forecasting," *Energies*, 2024.
- [2] Y. Li, L. Zhou, P. Gao, B. Yang, Y. Han, and C. Lian, "Short-term power generation forecasting of a photovoltaic plant based on pso-bp and ga-bp neural networks," *Frontiers in Energy Research*, 2022.
- [3] A. Ciaramella, A. Staiano, G. Cervone, and S. Alessandrini, "A bayesian-based neural network model for solar photovoltaic power forecasting," *Advances in Neural Networks*, 2015.
- [4] C. Zhang and M. Zhang, "Wavelet-based neural network with genetic algorithm optimization for generation prediction of pv plants," *Energy Reports*, 2022.
- [5] X. Gui, L. Tan, and Y. Huang, "A study of a photovoltaic power prediction method combining bilstm and gwo," in *International Conference Civil Engineering and Architecture*, 2024.
- [6] X. Wang, "Forecast of photovoltaic generated power based on woa-lstm," in *2020 5th International Conference on Mechanical, Control and Computer Engineering*, 2020.
- [7] D. Díaz-Bello, C. Vargas-Salgado, M. Alcázar-Ortega, and D. Alfonso-Solar, "Optimizing photovoltaic power plant forecasting with dynamic neural network structure refinement," *Scientific Reports*, 2025.
- [8] C. Deline, K. Perry, M. Deceglie, M. Muller, W. Sekulic, and D. Jordan, "Photovoltaic data acquisition (pvdaq) public datasets," 2021, public PV time-series database with system metadata and performance data.
- [9] T. Yao, J. Wang, H. Wu, P. Zhang, S. Li, Y. Wang, X.-b. Chi, and M. Shi, "A photovoltaic power output dataset: Multi-source photovoltaic power output dataset with python toolkit," *Solar Energy*, 2021.
- [10] S. Wimalaratne, D. Haputhanthri, S. Kahawala, G. Gamage, D. Alahakoon, and A. Jennings, "Unisolar: An open dataset of photovoltaic solar energy generation in a large multi-campus university setting," in *International Conference on Human System Interaction*, 2022.
- [11] D. Vázquez Pombo, O. Gehrke, and H. Bindner, "Solete, a 15-month long holistic dataset including meteorology, co-located wind and solar pv power," *Data in Brief*, 2022.
- [12] Y. Nie, X. Li, A. Scott, Y. Sun, V. Venugopal, and A. Brandt, "Skip'd: a sky images and photovoltaic power generation dataset for short-term solar forecasting," *Solar Energy*, 2023.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] A. Kannal, "Solar power generation data," Kaggle Datasets, 2020, accessed: 2026-04-21. [Online]. Available: <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>
- [15] C. e. a. Zhang, "Understanding generalization in deep learning," *ICLR*, 2017.
- [16] S. Mittal and J. Vetter, "Performance analysis of gpu acceleration for deep learning," *IEEE Transactions on Parallel and Distributed Systems*, 2020.
- [17] D. e. a. Sculley, "Trade-offs between accuracy and efficiency in machine learning," *NIPS*, 2015.
- [18] C. Deline, K. Perry, M. Deceglie, M. Muller, W. Sekulic, and D. Jordan, "Photovoltaic data acquisition (PVDAQ) public datasets," Open Energy Data Initiative (OEDI), NREL, 2021, accessed: 2026-04-20. [Online]. Available: <https://doi.org/10.25984/1846021>
- [19] C. Bergmeir and J. Benítez, "A note on the validity of cross-validation for evaluating time series prediction," *Journal of Forecasting*, 2018.
- [20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [21] A. Zheng and A. Casari, "Feature engineering for time series data," *O'Reilly*, 2018.
- [22] N. e. a. Srivastava, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, 2014.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015*, San Diego, CA, USA, 2015. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [24] J. Heaton, "Handling imbalanced data in neural networks," *Heaton Research*, 2017.
- [25] L. Prechelt, "Early stopping — but when?" *Neural Networks: Tricks of the Trade*, 1998.

- [26] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.
- [27] R. e. a. Poli, "Particle swarm optimization: Developments, applications and resources," *Swarm Intelligence*, 2007.
- [28] J. e. a. Chen, "On the reproducibility of neural network experiments," *arXiv preprint arXiv:1804.08775*, 2018.
- [29] S. e. a. Makridakis, "The m4 competition: Results, findings, conclusion," *International Journal of Forecasting*, 2020.
- [30] D. e. a. Yang, "Benchmarking solar forecasting models with persistence," *Solar Energy*, 2018.
- [31] K. Deb, "Multi-objective optimization using evolutionary algorithms," *Wiley*, 2001.
- [32] J. e. a. Antonanzas, "Solar power forecasting: A review," *Renewable and Sustainable Energy Reviews*, 2016.
- [33] D. e. a. Sculley, "Hidden technical debt in machine learning systems," *NIPS*, 2015.